

第 6 集 · 约 18 分钟

一次加密请求的完整旅程

跟着一条文字消息，从手机走到电脑，再走回来

配套阅读：书的第 6 章



这一集你会学到

1. 密钥藏在 # 后面：为什么它不会出现在网络上
2. http 局域网页面没有 WebCrypto，加密代码只能自己带：Rust 编译成的 WASM
3. 一条消息从点“发送”到界面刷新，每一层各做了什么
4. CSP：在响应头里写明“不执行别人的代码”
5. 页面刷新怎么办：只存密钥 K，每次重新握手

密钥就写在网址里

扫码时，密钥就写在网址里。同一个 WiFi 下抓包的人，能看到它吗？

看不到。密钥在 # 后面，浏览器根本不会把它发出去。

我们做出来的页面



电脑主界面（本机打开）。手机上是一个页面，只是排版不同

动手：# 后面去哪了

二维码里的网址（示例）

http:// 192.168.1.5 :8000 / #Hq3vV0n9...Kx

协议 · 电脑的局域网地址 · 端口 · 路径 · 片段：32 字节密钥

浏览器真正发出去的请求

GET / HTTP/1.1
Host: 192.168.1.5:8000
Accept: text/html

没有 # 后面的部分：密钥没有经过 WiFi

页面里的脚本

```
const key = keyFromFragment(location.hash);  
history.replaceState(null, "", location.pathname);  
  
读到密钥，立刻从地址栏抹掉
```

对比 v1（Python）：口令放在查询参数里

http://192.168.1.5:8000/?k=123456 → GET /?k=123456 HTTP/1.1
? 后面的部分会被原样发出去，同一个 WiFi 下抓包就能看到口令。

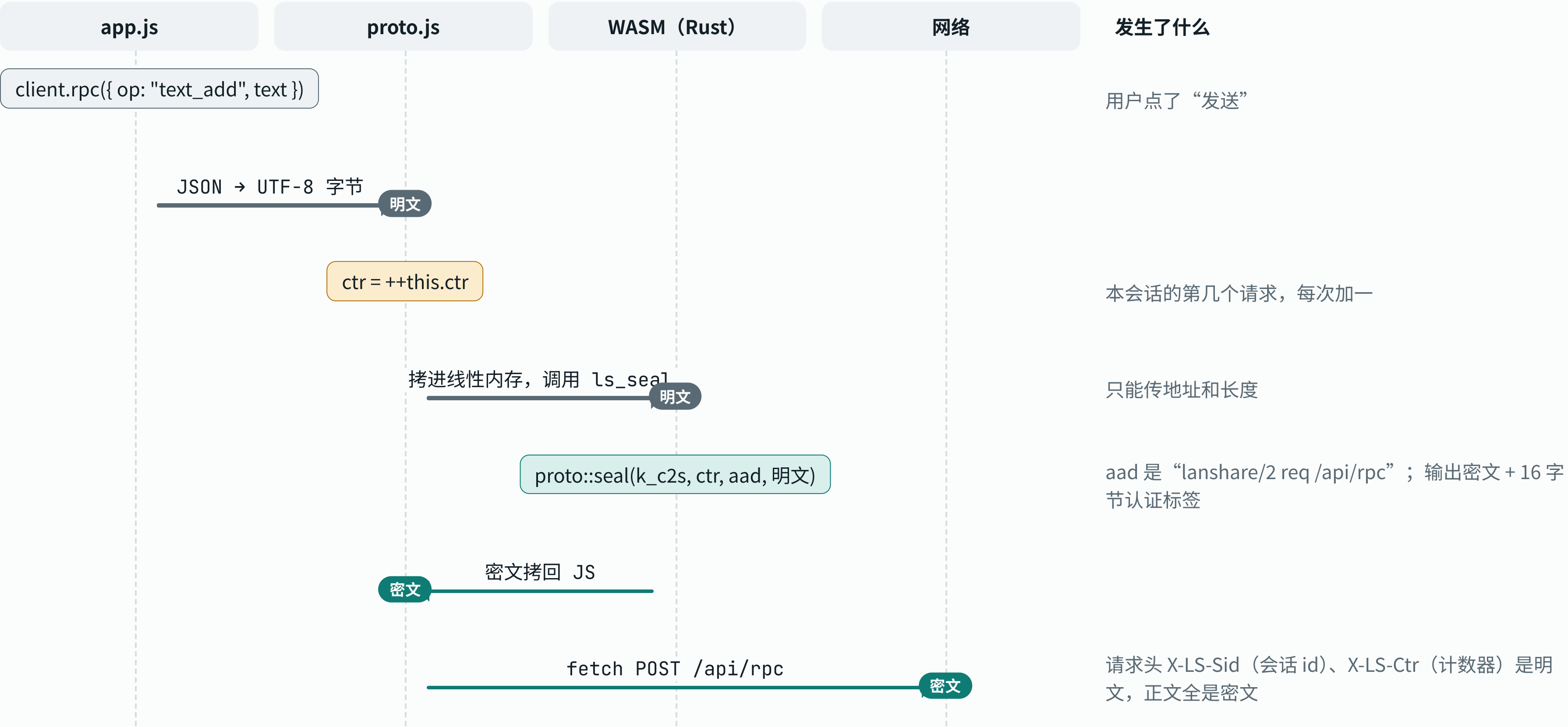
浏览器自带的加密呢？

```
Windows PowerShell

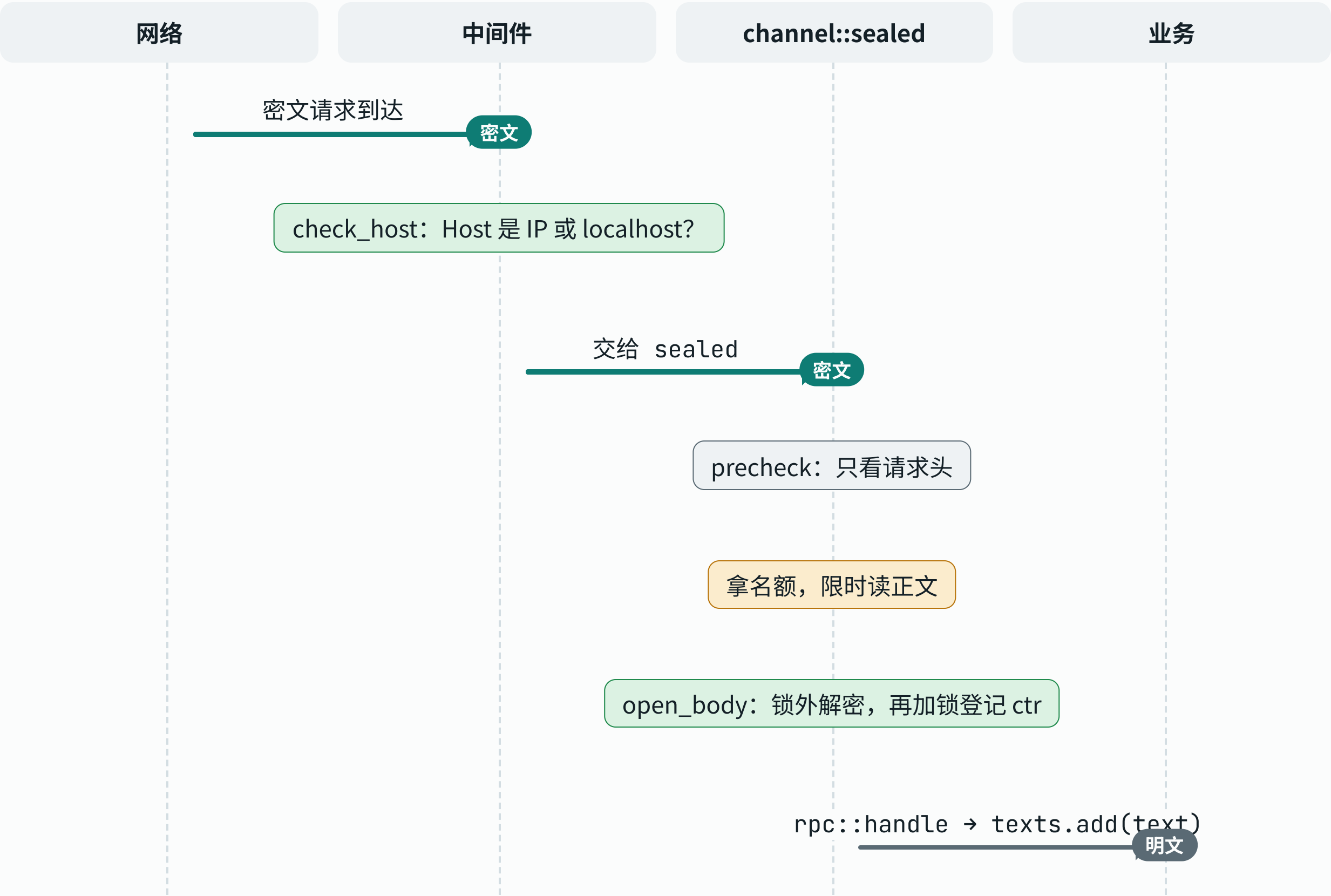
PS D:\lanshare> window.isSecureContext
false

PS D:\lanshare> window.crypto.subtle
undefined
```

第一段：手机上加密



第二段：电脑上验票、解密



发生了什么

连接先过准入：每个来源 IP 最多 16 个连接，请求头 10 秒内要收完

security_headers 给响应加 CSP 等安全头；
check_origin 挡跨站请求

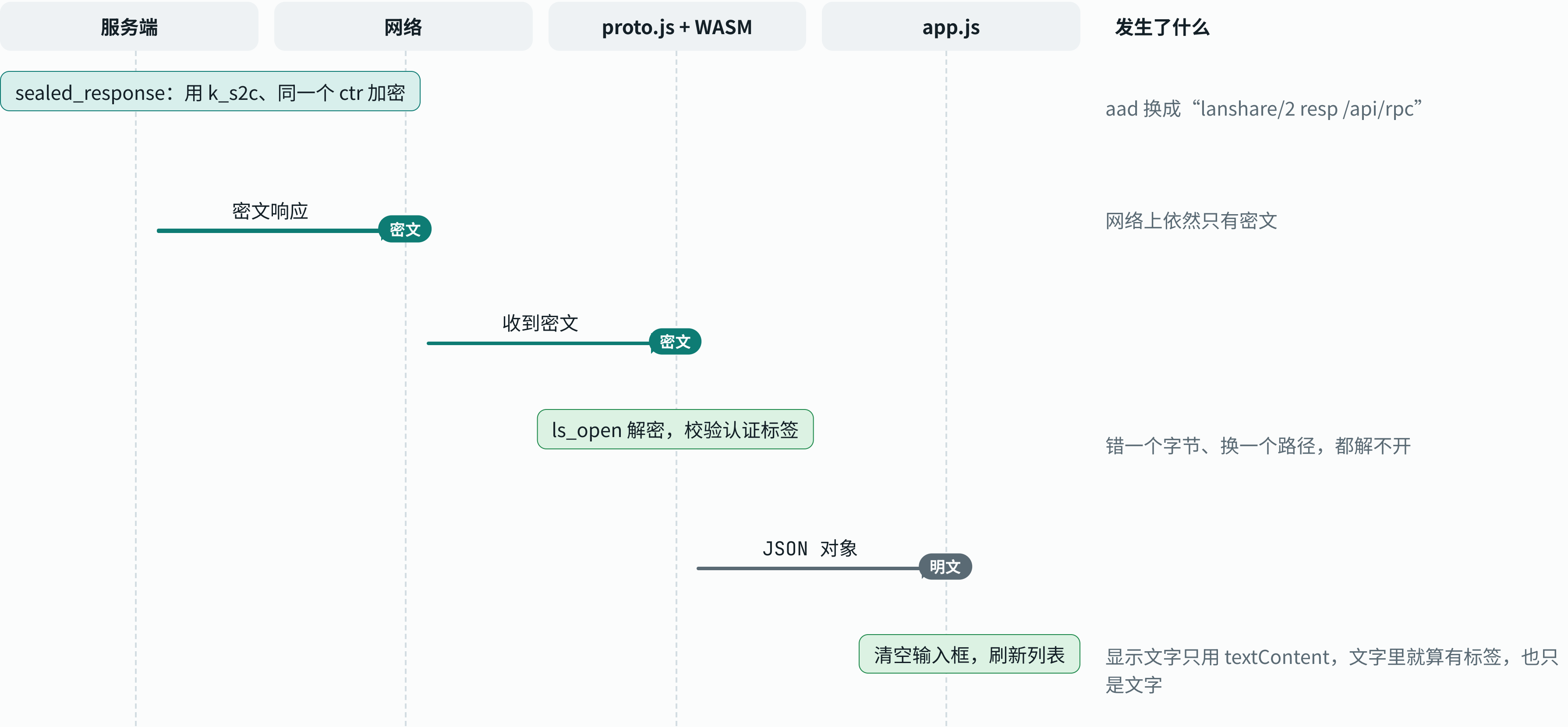
类型、长度、会话、计数器；正文一个字节都还没读

每个 IP 8 个、全局 32 个

aad 绑着路径，挪到别的接口就解不开

到这里，才第一次见到明文

第三段：原路加密返回



动手：改一个字节试试



演示用的是浏览器自带的 AES-GCM。LanShare 实际用 ChaCha20-Poly1305，两者都是“认证加密”：改一个字节、换一个附加数据都会解密失败，原理相同。nonce = 4 个零字节 + 8 字节计数器（和 LanShare 一样）。

v1 的口令是怎么漏的

Python v1 · server.py

```
1 def _index(self, query):
2     pin = parse_qs(query).get("k")          # 口令在 ?k=1234
3     if pin:
4         result, token = self.state.auth.try_login(self.cl
5         if result == OK:
6             self.send_response(303)
7             self.send_header("Location", "/")
8             self._send_cookie(token)        # Cookie 也是明文
```

Rust v2 · server/mod.rs

```
1 /// 二维码内容: 主地址 + `#密钥`。`#` 后面的部分浏览器不会发到网络
2 pub fn entry_url(&self) -> String {
3     format!("{}", self.base_urls()[0], self.key_fragmen
4 }
```

CSP: 不执行别人的代码

crates/lanshare/src/server/mod.rs

```
1  async fn security_headers(request: Request, next: Next) -> Response {
2      let mut response = next.run(request).await;
3      let h = response.headers_mut();
4      h.insert(
5          header::CONTENT_SECURITY_POLICY,
6          HeaderValue::from_static(
7              "default-src 'none'; script-src 'self' 'wasm-unsafe-eval'; style-src 'self'; img-src 'self' blob:; \
8              connect-src 'self'; base-uri 'none'; form-action 'none'; frame-ancestors 'none'",
9          ),
10 );
11 h.insert(header::X_CONTENT_TYPE_OPTIONS, HeaderValue::from_static("nosniff"));
12 h.insert(header::REFERRER_POLICY, HeaderValue::from_static("no-referrer"));
13 h.insert(header::X_FRAME_OPTIONS, HeaderValue::from_static("DENY"));
14 h.insert(header::CACHE_CONTROL, HeaderValue::from_static("no-store"));
15 response
16 }
```

这一章的真实踩坑

Node 的 spawn 不接受 URL 对象

用 new URL(...) 拼的 exe 路径直接传进去，报 ERR_INVALID_ARG_TYPE；要先用 fileURLToPath 转成字符串

node --test web-tests/ 不会扫描目录

它把目录当成一个模块去执行；要写成 node --test "web-tests/*.test.mjs"

真浏览器抓包里，搜到了 <svg

逐个定位，全是 index.html 里公开的图标；再确认真二维码的特征 crispEdges 在抓包里找不到

测试报警时，先查清楚，再下结论

选择题

为什么把密钥放在 `#` 后面，而不是 `?key=` 里？

A 因为浏览器会自动加密 `#` 后面的内容

B 两种都一样，反正 HTTP 都是明文

C 片段只留在浏览器里，不会被放进请求发出去

D 因为 `?key=` 后面放不下 32 字节的密钥

选择题

为什么把密钥放在 # 后面，而不是 ?key= 里？

A 因为浏览器会自动加密 # 后面的内容

浏览器不加密它，只是不把它发出去。真正的加密是我们自己的 WASM 做的。

B 两种都一样，反正 HTTP 都是明文

不一样。查询参数会上网络，片段不会。v1 的口令就是因为放在 ?k= 里才漏出去的。

C 片段只留在浏览器里，不会被放进请求发出去

对。?key= 属于查询参数，会出现在请求行里，抓包直接看到；# 后面的部分按规范不发出去。

D 因为 ?key= 后面放不下 32 字节的密钥

长度不是问题。问题在于它会不会被发到网络上。

选择题

把 CSP 里的 `'wasm-unsafe-eval'` 去掉，会怎样？

- A 没影响，WASM 是本站的文件，'self' 已经放行了
- B 只是 JS 的 eval 被禁了，其他照常
- C 页面照常能用，只是加密变慢
- D WASM 编译被拦，页面显示“加载加密模块失败”

选择题

把 CSP 里的 'wasm-unsafe-eval' 去掉，会怎样？

A 没影响，WASM 是本站的文件，'self' 已经放行了

'self' 管的是“从哪里加载脚本”，编译 WASM 还要单独许可。

B 只是 JS 的 eval 被禁了，其他照常

这条规则里本来就没放开 JS 的 eval。去掉它，受影响的正是 WASM。

C 页面照常能用，只是加密变慢

不是变慢，是根本加载不了。没有加密模块，页面什么都做不了。

D WASM 编译被拦，页面显示“加载加密模块失败”

对。WebAssembly 编译会被 CSP 拦下，app.js 加载失败时就显示这句话。它只放开 WASM 编译，不放开 JS 的 eval。

想一想

页面一刷新，proto.js 里的计数器就从 1 重新数。服务端会把它当成重放吗？如果让你设计，你会怎么办？

想一想

页面一刷新，proto.js 里的计数器就从 1 重新数。服务端会把它当成重放吗？如果让你设计，你会怎么办？

如果还用原来的会话，会：那些计数器已经用过了。把会话和计数器存进 localStorage 也不行，两个标签页同时用一个会话会撞号，刷新时机不巧也会撞。LanShare 的做法是只存二维码密钥 K，每次打开页面都换一个新的会话 id 重新握手。这样每个标签页、每次刷新都是独立的会话，计数器都从 1 开始，互不干扰。用口令登录的设备本来不知道 K，配对成功后服务端会经加密信道把 K 发给它，下次打开也不用再输口令。

这一集我们学了

- ❶ 密钥放在 # 后面：浏览器不发出去，页面拿到就从地址栏抹掉
- ❷ http 局域网页面没有 crypto.subtle，加密代码自己带：Rust 编译成 Wasm
- ❸ 一条消息：加密 → POST → 先验请求头 → 读正文 → 解密并登记 → 业务 → 反方向加密返回
- ❹ CSP 从浏览器层面兜底：不执行内联脚本，二维码当图片加载
- ❺ 页面刷新不存计数器，只存 K，每次换新会话重新握手

下一集

下一集：Windows 桌面外壳，托盘、只开一个、不弹黑窗口